

Lab 09 - Scheduling: nodeSelector, Affinity, Taints and Tolerations

Lab id	Lab 09
Day / Topic	Day 2 - Workloads & Scheduling
Duration	55 minutes
Namespace	kcna-lab09
Mapping	LO2 *Identify the technical and practical requirements in a Kubernetes setup* A3 *Identify technical and practical requirements as well as stakeholders' demands* K3 *Objectives of solution architecture*
Cluster	Single-node kind cluster, Kubernetes v1.30+

1. Objective

By the end of this lab you will be able to:

1. Describe scheduling as filter then score: hard constraints eliminate nodes, soft preferences only rank the survivors.
2. Place a Pod with nodeSelector and with nodeAffinity.requiredDuringSchedulingIgnoredDuringExecution, and explain what affinity buys you over nodeSelector.
3. Show that an unsatisfied preferredDuringSchedulingIgnoredDuringExecution term costs a node points but never blocks placement.
4. Add and remove a taint on a node, and write the matching toleration.
5. Read a FailedScheduling event and name the exact predicate that rejected every node.

Read this before you start - single-node reality

>

This cluster has one node. You cannot demonstrate placement by comparing node A with node B, so you will create the conditions yourself: you label the node, you taint the node, you remove the taint. Everything you do here is exactly what you would do on a 50-node cluster; only the node count differs.

>

Two consequences to keep in front of you:

>

* When a Pod goes Pending, the scheduler's message says `"0/1 nodes are available"`. On a production cluster it would read `"0/50 nodes are available"` with a breakdown per predicate. The mechanism is identical.

* kind removes the `node-role.kubernetes.io/control-plane:NoSchedule` taint on a single-node cluster, because otherwise nothing could ever run. Do not take that on trust - Step 1 checks it.

2. Prerequisites

```
kubectl version --output=yaml | grep gitVersion | head -1
kubectl get nodes
```

Expected output:

```
gitVersion: v1.31.0
NAME          STATUS    ROLES          AGE   VERSION
kind-control-plane  Ready    control-plane  68m   v1.31.0
```

You need permission to label and taint nodes (both are cluster-scoped writes). Confirm:

```
kubectl auth can-i patch nodes
```

Expected output:

```
yes
```

Work from the lab directory and capture the node name in a shell variable - every command below uses it, so the lab works whatever your cluster is called:

```
cd courseware/labs/lab-09-scheduling-affinity-taints
NODE=$(kubectl get nodes -o jsonpath='{.items[0].metadata.name}')
echo "NODE=$NODE"
```

Expected output:

```
NODE=kind-control-plane
```

3. Scenario

Marina Freight Systems (MFS) is preparing to move the *Harbour* platform onto hardware that is not uniform. The production cluster will have three classes of node:

- * sg-harbourfront nodes wired directly to the gate-scanner camera network,
- * SSD/NVMe nodes for the cargo-manifest store,
- * a small gpu-a100 pool that has been ordered but not yet delivered.

The platform lead has written data/node-placement-policy.csv - the contract between the workload owners and the platform team. Your task is to encode that contract in Kubernetes placement primitives and, crucially, to demonstrate what each one does when it cannot be satisfied, because half of the policy refers to hardware that does not exist yet.

You must also rehearse the monthly maintenance window: the node is tainted, batch work must stay off it, and the safety-critical crane controller must keep running.

4. Step-by-step procedure

Step 1 - Inspect the node you are about to steer

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab09 created
```

Look at the labels the node already has:

```
kubectl get node "$NODE" --show-labels
```

Expected output (one long line, wrapped here):

NAME	STATUS	ROLES	AGE	VERSION	LABELS
kind-control-plane	Ready	control-plane	69m	v1.31.0	beta.kubernetes.io/arch=arm64, beta.kubernetes.io/os=linux,kubernetes.io/arch=arm64,kubernetes.io/hostname=kind-control-plane, kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,node.kubernetes.io/exclude-from-external-load-balancers=

Every one of those is a legal nodeSelector target. Note `kubernetes.io/os=linux` - the eta-predictor Pod uses it as a hard floor later.

Now check the taint situation instead of assuming it.

```
kubectl get node "$NODE" -o jsonpath='{.spec.taints}'; echo " <-- empty means no taints"
```

Expected output on a single-node kind cluster:

```
<-- empty means no taints
```

```
kubectl describe node "$NODE" | grep -A2 '^Taints:'
```

Expected output:

```
Taints:          <none>
Unschedulable:   false
```

kind removed the control-plane taint when it built this single-node cluster. On a multi-node cluster (or a kubeadm cluster) you would instead see:

```
Taints:          node-role.kubernetes.io/control-plane:NoSchedule
```

and ordinary Pods would be kept off it. Check, do not assume - this is the single most common reason a lab "works on my cluster and not yours".

Step 2 - Read the placement policy and label the node from it

```
column -s, -t data/node-placement-policy.csv
```

Expected output:

workload	manifest	mechanism	
node_label_key	node_label_value	apply_label	toleration_key
toleration_effect	expected_phase	toleration_value	
gate-scanner	10-pod-nodeselector-gate-scanner.yaml	nodeSelector	mfs.
io/zone	sg-harbourfront	yes	
Running			
manifest-store	20-pod-affinity-required-manifest-store.yaml	requiredNodeAffinity	mfs.
io/disk	ssd	yes	
Running			
eta-predictor	30-pod-affinity-preferred-eta-predictor.yaml	preferredNodeAffinity	mfs.
io/zone	sg-tuas	no	
Running			
draft-optimiser	40-pod-affinity-unsatisfiable.yaml	requiredNodeAffinity	mfs.
io/hardware	gpu-a100	no	
Pending			
yard-report	50-pod-no-toleration-yard-report.yaml	none	
	no		
Pending			
crane-controller	60-pod-toleration-crane-controller.yaml	toleration	
	no	mfs.io/maintenance	window
			NoSchedule
Running			

The apply_label column tells you exactly which labels this node is supposed to carry. Two rows say yes - apply only those:

```
awk -F, 'NR>1 && $6=="yes" {print $4"="$5}' data/node-placement-policy.csv
```

Expected output:

```
mfs.io/zone=sg-harbourfront
mfs.io/disk=ssd
```

Apply them:

```
kubectl label node "$NODE" mfs.io/zone=sg-harbourfront mfs.io/disk=ssd
```

Expected output:

```
node/kind-control-plane labeled
```

Confirm - note that mfs.io/hardware and mfs.io/zone=sg-tuas are deliberately absent, because that hardware has not arrived:

```
kubectl get node "$NODE" -o jsonpath="zone={.metadata.labels['mfs.io/zone']} disk={.metadata.labels['mfs.io/disk']} hardware={.metadata.labels['mfs.io/hardware']}"; echo
```

Expected output:

```
zone=sg-harbourfront disk=ssd hardware=
```

```
kubectl get nodes -l mfs.io/zone=sg-harbourfront
kubectl get nodes -l mfs.io/hardware=gpu-a100
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane	71m	v1.31.0
No resources found				

One node satisfies the zone selector; zero satisfy the GPU selector. That second result is the whole of Step 6 in advance.

Step 3 - nodeSelector: the blunt instrument

```
grep -A2 'nodeSelector:' manifests/10-pod-nodeselector-gate-scanner.yaml
kubectl apply -f manifests/10-pod-nodeselector-gate-scanner.yaml
```

Expected output:

```
nodeSelector:
  mfs.io/zone: sg-harbourfront
containers:
pod/gate-scanner created
```

```
kubectl -n kcna-lab09 get pod gate-scanner -o wide
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
gate-scanner	1/1	Running	0	8s	10.244.0.12	kind-control-plane	<none>
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED							
gate-scanner 1/1 Running 0 8s 10.244.0.12 kind-control-plane <none>							

nodeSelector is a flat map of equality matches, ANDed. There is no In, no NotIn, no OR and no weighting. It is the right tool when the rule really is "this exact label".

Step 4 - Required node affinity: the same idea with operators

```
sed -n '/affinity:\/,/containers:/p' manifests/20-pod-affinity-required-manifest-store.yaml
```

Expected output:

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: mfs.io/disk
              operator: In
              values:
                - ssd
                - nvme
            - key: mfs.io/decommission
              operator: DoesNotExist
containers:
```

Read it precisely:

* nodeSelectorTerms are ORed - a node matching any one term is a candidate.

* matchExpressions inside a single term are ANDed - this node must have mfs.io/disk in {ssd, nvme} and must not carry mfs.io/decommission at all.

```
kubectl apply -f manifests/20-pod-affinity-required-manifest-store.yaml
kubectl -n kcna-lab09 get pod manifest-store -o wide
```

Expected output:

```
pod/manifest-store created
NAME          READY   STATUS    RESTARTS   AGE   IP           NODE
NOMINATED NODE READINESS GATES
manifest-store 1/1     Running   0           6s    10.244.0.13  kind-control-plane <none>
               <none>
```

The node has mfs.io/disk=ssd (in the value list) and no mfs.io/decommission key, so it survives both expressions.

The suffix IgnoredDuringExecution matters. If you now removed mfs.io/disk from the node, this Pod would keep running - node affinity is evaluated at scheduling time only. Prove it if you like, then re-apply the label:

>

```
```bash
```

```
kubectl label node "$NODE" mfs.io/disk-
```

```
kubectl -n kcna-lab09 get pod manifest-store
```

```
kubectl label node "$NODE" mfs.io/disk=ssd
```

```
...
```

>

Expected output:

>

```
...
```

```
node/kind-control-plane unlabeled
```

```
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
```

```
manifest-store 1/1 Running 0 70s 10.244.0.13 kind-control-plane <none> <none>
```

```
node/kind-control-plane labeled
```

```
...
```

## Step 5 - Preferred node affinity: a wish, not a rule

```
sed -n '/preferredDuringScheduling/,/containers:/p' manifests/30-pod-affinity-preferred-eta-predictor.yaml
```

Expected output:

```

 preferredDuringSchedulingIgnoredDuringExecution:
 - weight: 80
 preference:
 matchExpressions:
 - key: mfs.io/zone
 operator: In
 values:
 - sg-tuas
 - weight: 20
 preference:
 matchExpressions:
 - key: mfs.io/disk
 operator: In
 values:
 - ssd
 - nvme
 containers:

```

The sg-tuas zone does not exist on this cluster. The weight-80 preference therefore cannot be satisfied. Predict what happens, then:

```

kubectl apply -f manifests/30-pod-affinity-preferred-eta-predictor.yaml
kubectl -n kcna-lab09 get pod eta-predictor -o wide

```

Expected output:

```

pod/eta-predictor created
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED
NODE READINESS GATES
eta-predictor 1/1 Running 0 7s 10.244.0.14 kind-control-plane <none>
 <none>

```

It scheduled anyway. The node scored 20 out of a possible 100 on the affinity plugin and lost 80 points it could have had - but scoring only ranks candidates, it never eliminates them. The required term (kubernetes.io/os in (linux)) was satisfied, and that is the only part that could have blocked it.

This is the single most important distinction in the whole lab. Say it back before moving on:

Term	If satisfied	If not satisfied
requiredDuringSchedulingIgnoredDuringExecution	node stays a candidate	node eliminated - Pod may go Pending forever
preferredDuringSchedulingIgnoredDuringExecution	node gains weight points	node loses those points, still a candidate

## Step 6 - Taints: repel work from a node

The monthly maintenance window is starting. Batch jobs must stay off the node; the crane controller must not.

Apply the taint from the policy file:

```

awk -F, 'NR>1 && $7!="" {print $7"="$8":"$9}' data/node-placement-policy.csv

```

Expected output:

```

mfs.io/maintenance=window:NoSchedule

```

```
kubectl taint node "$NODE" mfs.io/maintenance=window:NoSchedule
```

Expected output:

```
node/kind-control-plane tainted
```

```
kubectl describe node "$NODE" | grep -A1 '^Taints:'
```

Expected output:

```
Taints: mfs.io/maintenance=window:NoSchedule
Unschedulable: false
```

First, confirm what a NoSchedule taint does not do:

```
kubectl -n kcna-lab09 get pods -o wide
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READYNESS	GATES				
eta-predictor	1/1	Running	0	3m	10.244.0.14	kind-control-plane
<none>	<none>					
gate-scanner	1/1	Running	0	8m	10.244.0.12	kind-control-plane
<none>	<none>					
manifest-store	1/1	Running	0	5m	10.244.0.13	kind-control-plane
<none>	<none>					

Nothing was evicted. NoSchedule affects *future* scheduling decisions only. The effect that evicts running Pods is NoExecute; the third effect, PreferNoSchedule, is a soft version that only lowers the node's score.

Now try to schedule a Pod with no toleration:

```
kubectl apply -f manifests/50-pod-no-toleration-yard-report.yaml
sleep 5
kubectl -n kcna-lab09 get pod yard-report
```

Expected output:

```
pod/yard-report created
NAME READY STATUS RESTARTS AGE
yard-report 0/1 Pending 0 5s
```

Read why:

```
kubectl -n kcna-lab09 describe pod yard-report | sed -n '/^Events:$/, $p'
```

Real output:



```
Events:
 Type Reason Age From Message
 ---- -
Warning FailedScheduling 12s default-scheduler 0/1 nodes are available: 1 node(s) had
untolaterated taint {mfs.io/maintenance: window}. preemption: 0/1 nodes are available: 1
Preemption is not helpful for scheduling.
```

The message names the taint, its value and how many nodes it eliminated. Now the crane controller, which tolerates it:

```
grep -A5 'tolerations:' manifests/60-pod-toleration-crane-controller.yaml
kubectl apply -f manifests/60-pod-toleration-crane-controller.yaml
kubectl -n kcna-lab09 get pod crane-controller -o wide
```

Expected output:

```
tolerations:
- key: mfs.io/maintenance
 operator: Equal
 value: window
 effect: NoSchedule
pod/crane-controller created
NAME READY STATUS RESTARTS AGE IP NODE
NOMINATED NODE READINESS GATES
crane-controller 1/1 Running 0 6s 10.244.0.16 kind-control-plane
<none> <none>
```

Side by side:

```
kubectl -n kcna-lab09 get pods -l 'placement in (no-toleration,toleration)' \
-o custom-columns=POD:.metadata.name,STATUS:.status.phase,NODE:.spec.nodeName,TOLERATES:.
spec.tolerations[0].key
```

Expected output:

POD	STATUS	NODE	TOLERATES
crane-controller	Running	kind-control-plane	mfs.io/maintenance
yard-report	Pending	<none>	<none>

A toleration is permission, not attraction. crane-controller was not \*drawn\* to the tainted node; it was merely \*allowed\* onto it. If you needed to both allow and steer, you would add a nodeSelector or nodeAffinity as well. Taints and node affinity are complementary controls, not alternatives - affinity is written by the \*workload\* owner ("I need SSD"), taints are written by the \*node\* owner ("stay off unless invited").

The maintenance window is over. Remove the taint - note the trailing minus:

```
kubectl taint node "$NODE" mfs.io/maintenance=window:NoSchedule-
sleep 5
kubectl -n kcna-lab09 get pod yard-report -o wide
```

Expected output:

node/kind-control-plane	untainted							
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	
NODE	READINESS	GATES						
yard-report	1/1	Running	0	2m	10.244.0.17	kind-control-plane	<none>	
	<none>							

The scheduler retried on its own - a Pending Pod stays in the scheduling queue and is re-evaluated whenever cluster state changes. Nothing was recreated; check the AGE column.

Re-apply the taint before running the verification script, because checks.sh asserts the maintenance-window end state:

```
kubectl -n kcna-lab09 delete pod yard-report
kubectl taint node "$NODE" mfs.io/maintenance=window:NoSchedule
kubectl apply -f manifests/50-pod-no-toleration-yard-report.yaml
```

Expected output:

```
pod "yard-report" deleted
node/kind-control-plane tainted
pod/yard-report created
```

---

## 5. Verification

```
bash verification/checks.sh; echo "exit=$?"
```

The script parses data/node-placement-policy.csv and, for every row, asserts (a) the node carries the labels whose apply\_label is yes, and (b) the Pod reached the expected\_phase in the policy.

Run it after section 6 and before section 8. The policy has six rows, and the sixth Pod (draft-optimiser) is only created in the failure-injection section. The script also requires the mfs.io/maintenance taint to still be applied, which is the end state of Step 6. Running it earlier will fail on the missing Pod and the missing taint - that is the script working, not the script being wrong.

Full transcript: [`verification/expected-output.md`](#).

---

## 6. Failure injection - an affinity that can never be satisfied

The GPU nodes were ordered in Q2 and have not shipped. The analytics team deploys against them anyway.

```
kubectl apply -f manifests/40-pod-affinity-unsatisfiable.yaml
```

Expected output:

```
pod/draft-optimiser created
```

```
sleep 10
kubectl -n kcna-lab09 get pod draft-optimiser
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
draft-optimiser	0/1	Pending	0	10s

Wait a minute and check again - it will still be Pending. This is a terminal state, not a transient one. Diagnose it:

```
kubectl -n kcna-lab09 describe pod draft-optimiser | sed -n '/^Events:$/, $p'
```

Real output:

```
Events:
 Type Reason Age From Message
 ---- -
Warning FailedScheduling 63s default-scheduler 0/1 nodes are available: 1
node(s) didn't match Pod's node affinity/selector. preemption: 0/1 nodes are available: 1
Preemption is not helpful for scheduling.
```

The scheduling condition:

```
kubectl -n kcna-lab09 get pod draft-optimiser \
-o jsonpath='{range .status.conditions[*]}{.type}={.status} reason={.reason} msg={.message}}{
"\n"}{end}'
```

Expected output:

```
PodScheduled=False reason=Unschedulable msg=0/1 nodes are available: 1 node(s) didn't match
Pod's node affinity/selector. preemption: 0/1 nodes are available: 1 Preemption is not helpful
for scheduling.
```

And no node was ever assigned:

```
kubectl -n kcna-lab09 get pod draft-optimiser -o jsonpath='{.spec.nodeName}'; echo
```

Expected output:

```
nodeName=[]
```

Diagnosis - read the message in three parts.

1. 0/1 nodes are available - the scheduler considered every node in the cluster. One node, zero survivors. On a 50-node cluster this would read 0/50 with a per-reason tally.
2. 1 node(s) didn't match Pod's node affinity/selector - the *\*filter\** phase eliminated it. This is a hard constraint failing, so no amount of waiting will help.
3. preemption: ... Preemption is not helpful - the scheduler also checked whether evicting lower-priority Pods would free up a *\*suitable\** node. It would not, because the node is not suitable at all - it is missing a label, not short of capacity. That last clause is how you distinguish "wrong node" from "full node". A capacity failure instead says Insufficient cpu or Insufficient memory.

Find the offending term yourself:

```
kubectl -n kcna-lab09 get pod draft-optimiser \
-o jsonpath='{.spec.affinity.nodeAffinity.requiredDuringSchedulingIgnoredDuringExecution.
nodeSelectorTerms[0].matchExpressions[0]}' ; echo
kubectl get nodes -l mfs.io/hardware=gpu-a100
```

Expected output:

```
{"key":"mfs.io/hardware","operator":"In","values":["gpu-a100"]}
No resources found
```

Fix - pick one, and understand the difference.

\*Option A: the hardware genuinely arrived, so label the node.\* (In this lab that would be a lie, so undo it afterwards.)

```
kubectl label node "$NODE" mfs.io/hardware=gpu-a100
sleep 5
kubectl -n kcna-lab09 get pod draft-optimiser -o wide
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READYNESS	GATES				
draft-optimiser	1/1	Running	0	3m	10.244.0.18	kind-control-plane
<none>						<none>

The Pod was never recreated - its AGE is unchanged. It sat in the scheduling queue and was placed the moment the cluster could satisfy it.

Restore the honest state before verifying:

```
kubectl -n kcna-lab09 delete pod draft-optimiser
kubectl label node "$NODE" mfs.io/hardware-
kubectl apply -f manifests/40-pod-affinity-unsatisfiable.yaml
```

Expected output:

```
pod "draft-optimiser" deleted
node/kind-control-plane unlabeled
pod/draft-optimiser created
```

\*Option B: downgrade the requirement to a preference.\* This is what the analytics team should have written, because the model runs on CPU too - just slower:

```
sed 's/requiredDuringSchedulingIgnoredDuringExecution:/
preferredDuringSchedulingIgnoredDuringExecution:/' \
manifests/40-pod-affinity-unsatisfiable.yaml | kubectl diff -f - | head -20
```

This prints the diff without applying it, so the Pending evidence stays intact for the verification step. A hard requirement for hardware you do not own is the most common self-inflicted Pending in production.

---

## 7. Troubleshooting

Symptom	Cause	Fix
0/N nodes are available: N node(s) didn't match Pod's node affinity/selector.	A nodeSelector or a required nodeAffinity term matches no node	kubectl get nodes -l <key>=<value> to confirm; label a node, or downgrade the term to preferred
0/N nodes are available: N node(s) had untoleration taint {key: value}.	The node is tainted and the Pod has no matching toleration	Add a toleration with matching key, value and effect, or remove the taint with <code>kubectl taint node &lt;n&gt; key=value:Effect-</code>
0/N nodes are available: N Insufficient cpu.	Capacity, not placement - the node's allocatable CPU is already committed	Lower resources.requests, scale down other workloads, or add a node (see Lab 10)
Pod is Pending with no events at all	Events older than roughly one hour are garbage-collected	<code>kubectl -n &lt;ns&gt; delete pod &lt;p&gt;</code> and re-apply to regenerate the event, or read <code>.status.conditions[?(@.type=="PodScheduled")].message</code> which never expires
A preferred affinity term is ignored	Working as designed - preferences only contribute to the node score	If the placement is mandatory, move the term to <code>requiredDuringSchedulingIgnoredDuringExecution</code>
Pod keeps running after you delete the node label it required	<code>IgnoredDuringExecution</code> - node affinity is evaluated only at scheduling time	Use a <code>NoExecute</code> taint if you need running Pods evicted; there is no <code>RequiredDuringExecution</code> affinity
error: at least one taint update is required	The kubectl taint argument was malformed - the effect or the trailing - is missing	Add: <code>key=value:NoSchedule</code> ; remove: <code>key=value:NoSchedule-</code>
Tainting the node makes system Pods (CoreDNS, CNI) go Pending too	On a single-node cluster your taint applies to the only node, and add-ons without that toleration are affected	Untaint promptly: <code>kubectl taint node &lt;n&gt; mfs.io/maintenance=window:NoSchedule-</code> ; never leave a lab taint in place

---

## 8. Cleanup

Delete only this lab's namespace - and then revert the two cluster-scoped changes you made to the node. Node labels and taints live outside the namespace, so deleting the namespace does not remove them, and leaving a `NoSchedule` taint behind will break every lab that follows.

```
kubectl delete namespace kcna-lab09
```

Expected output:

```
namespace "kcna-lab09" deleted
```

```
NODE=$(kubectl get nodes -o jsonpath='{.items[0].metadata.name}')
kubectl taint node "$NODE" mfs.io/maintenance=window:NoSchedule-
kubectl label node "$NODE" mfs.io/zone- mfs.io/disk-
```

Expected output:

```
node/kind-control-plane untainted
node/kind-control-plane unlabeled
```

Confirm the node is back to how you found it:

```
kubectl get node "$NODE" -o jsonpath='{.spec.taints}'; echo
kubectl get node "$NODE" --show-labels | grep -c 'mfs.io' || echo "no mfs.io labels remain"
kubectl get ns kcna-lab09
```

Expected output:

```
taints=
no mfs.io labels remain
Error from server (NotFound): namespaces "kcna-lab09" not found
```

---

## 9. What you learned

- \* Scheduling is filter then score. Filtering (predicates) eliminates nodes; scoring ranks whatever survives. A Pod goes Pending only when filtering leaves zero nodes.
- \* nodeSelector is a flat AND of equality matches. nodeAffinity does the same job with operators (In, NotIn, Exists, DoesNotExist, Gt, Lt), OR across nodeSelectorTerms and AND within matchExpressions.
- \* requiredDuringSchedulingIgnoredDuringExecution can leave a Pod Pending forever. preferredDuringSchedulingIgnoredDuringExecution never can - it only moves points.
- \* IgnoredDuringExecution means the constraint is checked once, at scheduling time. Removing the label later does not evict the Pod.
- \* Taints repel, tolerations permit. NoSchedule blocks new placements, NoExecute also evicts running Pods, PreferNoSchedule merely lowers the score. A toleration never *attracts* a Pod - pair it with affinity when you also need to steer.
- \* Node affinity is written by the workload owner; taints are written by the node owner. They are complementary, not interchangeable.
- \* The FailedScheduling message is a complete diagnosis: node tally, failing predicate, and whether preemption would help. Preemption is not helpful means the node is *unsuitable*; Insufficient cpu means it is merely *full*.
- \* Node labels and taints are cluster-scoped. Deleting a namespace does not undo them - clean them up explicitly.

## 10. Further reading

- \* Assigning Pods to Nodes - <<https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/>>
- \* Taints and Tolerations - <<https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>>
- \* Kubernetes Scheduler - <<https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>>
- \* Pod Priority and Preemption - <<https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/>>
- \* Well-Known Labels, Annotations and Taints - <<https://kubernetes.io/docs/reference/labels-annotations-taints/>>
- \* Pod Topology Spread Constraints - <<https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>>